

A Graph-Based Semisupervised Deep Learning Model for PolSAR Image Classification

Haixia Bi[✉], Jian Sun[✉], *Member, IEEE*, and Zongben Xu

Abstract—Aiming at improving the classification accuracy with limited numbers of labeled pixels in polarimetric synthetic aperture radar (PolSAR) image classification task, this paper presents a graph-based semisupervised deep learning model for PolSAR image classification. It models the PolSAR image as an undirected graph, where the nodes correspond to the labeled and unlabeled pixels, and the weighted edges represent similarities between the pixels. Upon the graph, we design an energy function incorporating a semisupervision term, a convolutional neural network (CNN) term, and a pairwise smoothness term. The employed CNN extracts abstract and data-driven polarimetric features and outputs class label predictions to the graph model. The semisupervision term enforces the category label constraints on the human-labeled pixels. The pairwise smoothness term encourages class label smoothness and the alignment of class label boundaries with the image edges. Starting from an initialized class label map generated based on K -Wishart distribution hypothesis or superpixel segmentation of PauliRGB images, we iteratively and alternately optimize the defined energy function until it converges. We conducted experiments on two real benchmark PolSAR images, and extensive experiments demonstrated that our approach achieved the state-of-the-art results for PolSAR image classification.

Index Terms—Convolutional neural network (CNN), graph model, polarimetric synthetic aperture radar (PolSAR) image classification, semisupervised method.

I. INTRODUCTION

AS ONE of the most prominent techniques in microwave remote sensing fields, polarimetric synthetic aperture radar (PolSAR) has obtained remarkable progress in recent years. Massive PolSAR images have been acquired by the launched polarimetric sensors all over the world. PolSAR image classification task aims to divide the images into different terrain categories, such as water, vegetation, and urban area, which plays an important role in city planning, sea monitoring, geological exploration, and plant growth status assessment [1]. A great variety of PolSAR image classification methods have been proposed in the past decade [1]–[20]. Considering whether the data are labeled and how the labeled

data are used in the classification, these methods can be classified into three categories: supervised methods, unsupervised methods, and semisupervised ones.

Supervised methods [2]–[11] use a number of labeled pixels to train a classifier, and then, the learned classifier is used for predicting the remaining pixels. A great deal of supervised algorithms have been used in PolSAR image classification, including Bayesian classification method [2], neural network [3], [4], support vector machine (SVM) [5]–[7], and k -nearest neighbor [8]. Recently, deep learning approach has been introduced to PolSAR image classification task and achieved promising performance [9]–[11]. Supervised methods can achieve high classification accuracy (CA), and however, they rely heavily on large numbers of labeled data, which require great efforts of experienced human annotators.

Unsupervised methods [1], [12]–[20] try to divide pixels to a number of categories based on polarimetric statistical rules without using any labeled data. Polarimetric target decomposition theories, such as Cloude–Pottier decomposition [12], Freeman decomposition [13], and four-component decomposition [14], are commonly used to generate an initial segmentation. And then, iterative algorithms based on diverse polarimetric statistical laws, such as complex Wishart distribution [1], K -distribution [15], [16], and $\mathcal{U}$ -distribution [17], are employed to update the classification result [18]–[20]. Unsupervised methods are fast and simple. However, their results are mostly imprecise, which makes them only suitable for preliminary analysis of PolSAR data.

Since it is labor intensive and time-consuming to obtain sufficient labeled pixels for PolSAR images, how to achieve a favorable performance using limited numbers of labeled pixels becomes a challenging task. The semisupervised method provides a possible solution for this problem. It focuses on improving CA and generalization ability using both the labeled and unlabeled pixels. The semisupervised method incorporates the advantages of supervised and unsupervised methods, and therefore, it is particularly applicable for image classification tasks with few labeled pixels and a large number of unlabeled pixels. The semisupervised image classification problem has received increasing interests and evolved rapidly recently [21]–[25].

The graph-based semisupervised method is a representative semisupervised learning approach that attracts extensive attentions due to its solid mathematical foundation [24], [25] and an excellent representation of inherent structure between the pixels. In the graph-based semisupervised image classification

Manuscript received December 16, 2017; revised April 20, 2018 and August 11, 2018; accepted September 15, 2018. This work was supported by the National Natural Science Foundation of China under Grant 11622106, Grant 11690011, Grant 61472313, and Grant 61721002. (Corresponding author: Jian Sun.)

The authors are with the Institute for Information and System Sciences and the National Engineering Laboratory for Big Data Algorithm and Analysis Technology, Xi'an Jiaotong University, Xi'an 710049, China (e-mail: bhxwzq@163.com; jiansun@xjtu.edu.cn; zbxu@mail.xjtu.edu.cn).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TGRS.2018.2871504

method, a graph is first constructed on an image, which consists of nodes corresponding to pixels and weighted edges representing the similarities between the pixels. Then, the class information spreads through edges from the labeled pixels to the unlabeled ones, which is mostly implemented using a designed optimization algorithm. Several works have investigated the graph-based semisupervised method for PolSAR image classification in the past few years [26]–[29]. Please refer to Section II-A for more discussions on these methods. Although promising advances have been achieved, how to explore more representative polarimetric features and realize effective class label propagation (LP) based on the graph is still a challenging task.

In this paper, we propose a semisupervised PolSAR image classification method that combines feature learning based on deep neural network and graph-based class label propagation in a principled framework. The main novelties and contributions of this paper are summarized as follows.

- 1) We introduce a graph-based semisupervised deep learning model to PolSAR image classification. Specifically, we design an energy function combining a semisupervision term, a convolutional neural network (CNN) term, and a pairwise smoothness term. The semisupervision term enforces class label constraints on the human-labeled pixels. The CNN term is responsible for extracting data-driven polarimetric features and predicting class possibilities. The pairwise smoothness term encourages class label smoothness and the alignment of class label boundaries with the image edges. Both the CNN parameters and class labels are unknown variables to be optimized.
- 2) We propose an iterative and alternating algorithm to optimize the CNN parameters and class labels. In each iteration, we first optimize the CNN parameters by learning a deep neural network and then optimize the class labels through graph-based label propagation. These two tasks are dependent on each other. The CNN is supervised by the propagated class labels and, in turn, provides class label predictions for the following label propagation. Initialized from a class label map generated based on K -Wishart distribution hypothesis or superpixel segmentation of PauliRGB images, the model is optimized iteratively and alternately until it converges.
- 3) To demonstrate the effectiveness of the proposed method, we conduct experiments on two real benchmark PolSAR images. Experimental results evidently show the advantages of our proposed method compared with other state-of-the-art PolSAR image classification methods.

To the best of our knowledge, this is the first work that incorporates graph-based semisupervised classification with deep neural network into a unified PolSAR image classification framework. The remainder of this paper is organized as follows. Section II gives a brief review of the related works on semisupervised and deep learning-based PolSAR image classification methods. We describe the proposed graph-based semisupervised deep learning model in Section III. Section IV reports experiments on two real benchmark PolSAR images. Conclusions and future works are discussed in Section V.

II. RELATED WORKS

A. Semisupervised PolSAR Image Classification

Since it is difficult to acquire labeled data for the PolSAR image, semisupervised learning lights a promising way for PolSAR image classification. Several semisupervised PolSAR image classification methods have been proposed, achieving favorable performances [26]–[31].

In the literature of semisupervised PolSAR image classification, the majority of the methods are based on the graph models. Wei *et al.* [26] presented a hypergraph-based algorithm, which utilizes a hypergraph to express the complex relationship among the interested objects in PolSAR images. A semisupervised method based on a spatial-anchor graph was proposed by Liu *et al.* [27]. In this method, an initial segmentation is conducted to obtain spatially homogeneous superpixel, and then, unsupervised Wishart clustering is performed to select the cluster centers as anchors, based on which the spatial-anchor graph is constructed. Finally, class information propagates from the labeled pixels to the unlabeled ones. Afterward, Liu *et al.* [28] put forward a fast semisupervised classification method based on the spatial-anchor graph, which improves [27] in superpixel segmentation and class label assignment aspects for pixels on the border of the superpixels.

Besides, several other PolSAR image classification methods were proposed based on other semisupervised models. Hansch and Hellwich [30] presented a semisupervised learning scheme, where the data structure is first analyzed by clustering technique, and the association potential of a data point to clusters is used to train multilayer perceptrons, which automatically adapts the number of input neurons to the current number of clusters. A self-training approach using an ensemble-based classifier was proposed in [31] for PolSAR image classification. Different strategies on how to select more reliable candidates from the pool of unlabeled pixels to speed up the learning process are learned in this paper.

Semisupervised learning has also been employed in polarimetric feature extraction. A semisupervised feature extraction method with neighborhood constraints was proposed in [29], which extracts discriminative and structure-preserving features of PolSAR data. The redundancy among the features is reduced, and separability of features is validated.

B. Deep-Learning-Based PolSAR Image Classification

Deep learning, as a hierarchical feature learning approach, has been widely utilized in image processing and analysis [32]–[37]. The research and application of deep learning in remote sensing have been studied in several works [38]–[41]. Compared with traditional methods using artificial features [7], [42]–[46], [48], CNN is able to automatically extract effective discriminative features for a specific task. In addition, deep learning-based models have the unique capacity to hierarchically encode spatial feature information.

The deep learning approach provides a new way for PolSAR image classification. Several advanced deep learning architectures have been employed in PolSAR image

classification recently. Liu *et al.* [9] proposed a PolSAR image classification method based on deep belief network (DBN), where a Wishart DBN (WDBN) stacked by Wishart–Bernoulli restricted boltzmann machines (WBRBMs) and binary RBM is used. Moreover, local spatial information is incorporated to clean the preliminary classification results generated by WDBN. Stacked sparse autoencoder considering local spatial information was employed to learn deep spatial sparse features of PolSAR data by Zhang *et al.* [45]. The speckle noise embedded in a PolSAR imagery process is demonstrated to be reduced, and improved accuracy and good visual coherence can be observed. Jiao and Liu [10] designed a PolSAR image classification method based on deep stacking network (DSN), which establishes a novel connection between PolSAR image interpretation and deep learning. Experimental results proved that the Wishart-DSN (WDSN) is an effective and efficient deep architecture specialized for PolSAR image classification. Zhou *et al.* [11] proposed a CNN-based PolSAR image classification method, where hierarchical polarimetric spatial features can be automatically learned from the PolSAR image classification task.

For this application, our proposed method differs from the above-mentioned semisupervised and deep learning-based PolSAR image classification methods in three perspectives. First, instead of using semisupervised learning or deep learning individually, our proposed method combines the two components into a unified framework. Second, contextual information is fully exploited owing to the hierarchically encoded spatial information realized by CNN and the incorporation of the pairwise smoothness constraint. Besides, the defined pairwise smoothness term encourages the alignment of class label boundaries with the image edges. Finally, unlike [27] and [52] that build graph models on superpixels, we establish pixelwise graphs on PolSAR images, in order to get detail-preserving classification results.

III. METHODOLOGY

A. Overview of Our Method

In the following, we will first define some notations and then introduce the framework of the proposed method. For a given PolSAR image, the PolSAR feature data are defined as $X \in \mathbb{R}^{D_1 \times D_2 \times D_3}$, where D_1 and D_2 are the height and width of the PolSAR image, and D_3 is the dimension of the selected raw feature vector. Then, the total pixel number of the image N equals $D_1 \times D_2$. The labeled pixel pairs are denoted as $L = \{(x_1 \ t_1); (x_2 \ t_2); \dots; (x_n \ t_n)\}$, where $n \ll N$, $t_i \in \{1, 2, \dots, K\}$, $x_i \in S_{\text{sup}}$. Here, K is the total number of classes, and S_{sup} is the labeled pixel set. The proposed method is designed to assign class label y_i for each pixel i ($i \in \{1, 2, \dots, N\}$). We further denote $Y = \{y_i\}$ in Section III.

Fig. 1 shows the framework of our proposed method, which formulates graph-based semisupervision and deep neural network into a unified model. We establish a graph on a given PolSAR image, which can be regarded as a Markov random field (MRF). The supervision imposes constraints from randomly selected human-labeled pixels and the CNN outputs

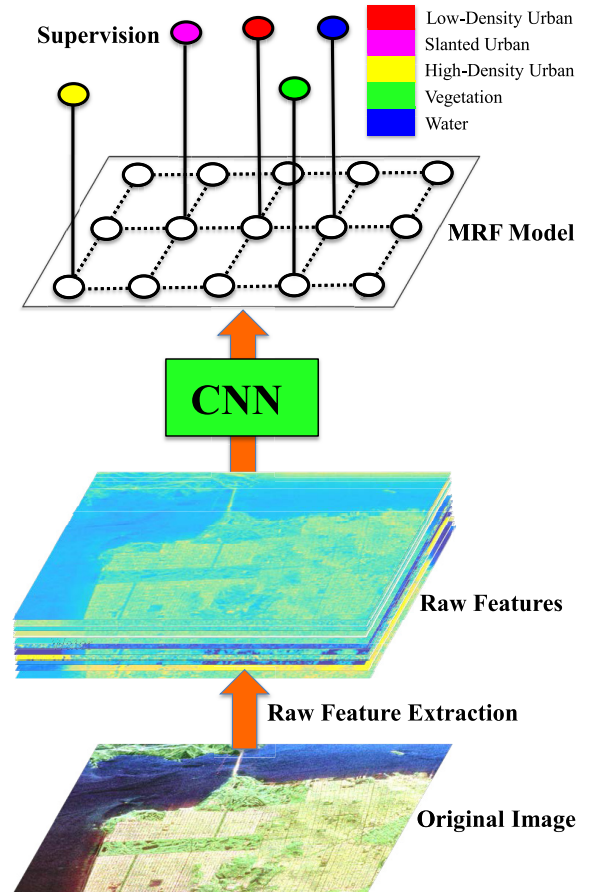


Fig. 1. Overview of our graph-based semisupervised deep learning model for PolSAR image classification. The graph is built on the PolSAR image, which can be regarded as an MRF. The supervision imposes constraints from human-labeled pixels. The CNN outputs class label predictions to the MRF. The embedded pairwise smoothness prior in MRF is used to enforce the smoothness of the class labels over the graph.

pixel predictions to the MRF. The embedded pairwise smoothness prior in MRF encourages class label smoothness during the class label propagation. We define an energy function, which incorporates the semisupervision, CNN predictions, and pairwise smoothness prior together. An iterative optimization algorithm is designed to minimize the energy function. The initialized class label map for the optimization is generated based on the K -Wishart distribution hypothesis or superpixel segmentation of PauliRGB images, and the raw features are extracted as input for the CNN.

In Section III, we first introduce the extracted feature sets from the PolSAR image as the input raw features of our approach. Second, we propose the graph-based semisupervised deep learning energy function for feature learning and class label prediction. Third, we design an iterative optimization algorithm for energy minimization by alternately updating network parameters and pixelwise class labels. To initialize this iterative algorithm, we introduce two initialization methods to generate the initial class labels. Finally, we summarize our proposed method.

TABLE I
RAW POLARIMETRIC FEATURES EMPLOYED
IN THE PROPOSED METHOD

Designation	Description
RF-1 = $10\log_{10}(\text{SPAN})$	Polarimetric total power in decibel
RF-2 = T_{22}/SPAN	Normalized ratio of power T_{22}
RF-3 = T_{33}/SPAN	Normalized ratio of power T_{33}
RF-4 = $ T_{12} /\sqrt{T_{11} \cdot T_{22}}$	Relative correlation coefficient of T_{12}
RF-5 = $ T_{13} /\sqrt{T_{11} \cdot T_{33}}$	Relative correlation coefficient of T_{13}
RF-6 = $ T_{23} /\sqrt{T_{22} \cdot T_{33}}$	Relative correlation coefficient of T_{23}

B. Raw Features Extraction

We first extract raw polarimetric features as input of our approach. As shown in Table I, our extracted polarimetric feature is a 6-D real vector representation of PolSAR data (denoted as RF- i , $i = 1, \dots, 6$). The employed features are all extracted from the second-order 3×3 complex coherency polarimetric matrix T , which is based on the Pauli basis of scattering matrix of PolSAR [12].

In the employed features, RF-1 is the decibel format of the total polarimetric power SPAN ($\text{SPAN} = T_{11} + T_{22} + T_{33}$). RF-2 and RF-3 denote the normalized ratios of the powers T_{22} and T_{33} , respectively. RF-4–RF-6 are the relative correlation coefficients of the cross polarization components T_{12} , T_{13} , and T_{23} . Features RF-2–RF-6 are normalized in $[0, 1]$. The raw features we use in this paper are the same as [11].

C. Graph-Based Semisupervised Deep Learning Model

Given the extracted raw features as input, we next introduce our proposed graph-based semisupervised deep learning model. We establish a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ over a given PolSAR image, where $\mathcal{V}$ is the node set corresponding to pixels, and $\mathcal{E}$ is the weighted edge set representing pixel similarities. Then, we define class label y_i as a variable on each node $v_i \in \mathcal{V}$. We define the following energy function for semisupervised classification:

$$E(Y, F_\Theta | X, L) = E_s(Y | X, L) + E_c(F_\Theta | X, Y) + E_p(Y | X) \quad (1)$$

where F_Θ is the nonlinear transform of CNN with parameter Θ , $E_s(Y | X, L)$ indicates the *semisupervision term*, $E_c(F_\Theta | X, Y)$ is the *CNN term*, and $E_p(Y | X)$ is the *pairwise smoothness term*. By minimizing this energy function, we aim to estimate the dense class labels Y and learn the CNN F_Θ with parameter set Θ .

1) *Semisupervision Term*: The semisupervision term enforces the class label constraints on the human-labeled pixels. For the labeled pixels, it strongly encourages that the estimated class labels should be the same with the ground-truth class labels. While for the unlabeled pixels, it has no preference for the class label prediction.

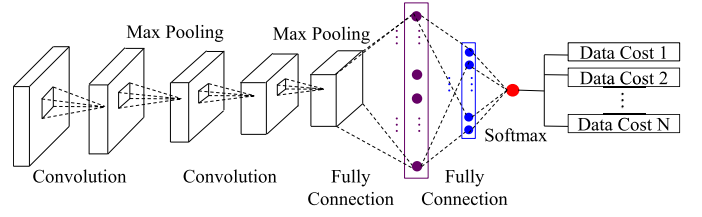


Fig. 2. Utilized CNN architecture for PolSAR image classification. It is comprised of two convolutional layers followed by max pooling, two fully connected layers, and a softmax output layer.

The semisupervision term is defined based on the randomly selected human-labeled pixel set L , which is defined as

$$E_s(Y | X, L) = \sum_{i=1}^N \psi_i^{\text{semi}}(y_i | X, L) \quad (2)$$

where $\psi_i^{\text{semi}}(y_i | X, L)$ is the semisupervision term for each pixel, which is defined as

$$\psi_i^{\text{semi}}(y_i | X, L) = \begin{cases} 0, & \text{if } x_i \in S_{\text{sup}} \text{ and } y_i = t_i \\ \infty, & \text{if } x_i \in S_{\text{sup}} \text{ and } y_i \neq t_i \\ -\log\left(\frac{1}{K}\right), & \text{otherwise.} \end{cases} \quad (3)$$

For a labeled pixel x_i , if the class label prediction y_i is the same as the ground-truth class label t_i , the semisupervision cost equals zero; otherwise, it has cost of infinity. For an unlabeled pixel, it can be assigned with any class label present in the image, so the cost is set as an equal log probability for all class label predictions. For a labeled pixel, minimizing this term means that, during the class label propagation process, the class label is strongly preferred to be the same as the ground-truth class label, which is greatly beneficial for reducing false positive predictions.

2) *CNN Term*: The CNN term is designed to predict class label based on the learned discriminative features. As shown in Fig. 2, we build a CNN comprising two groups of convolutional layer and pooling layer, followed by two fully connected layers and a softmax classifier connected to the output. For PolSAR image classification task, the input for each pixel x_i is a 3-D patch. We will detail the process of how each layer in the proposed CNN structure works on the input patch in the following paragraph.

The convolutional layer convolves feature maps with a set of learnable filters. For the first convolutional layer, the feature maps are taken as the extracted raw features (discussed in Section III-B), and for the consequent layers, they are taken as the output of the preceding pooling layer. The output feature map after the convolution operation can be expressed as

$$\alpha_j^l = \sigma \left(\sum_{i=1}^{m_c} x_i^{(l-1)} * W_{ji}^l + b_j^l \right) \quad (4)$$

where α_j^l is the j th feature map of the l th layer and $x_i^{(l-1)}$ is the i th input feature map of the $(l-1)$ th layer. m_c is the number of the feature maps fed into the convolutional layer. $\sigma(\cdot)$ is the nonlinear activation function, which is taken as a rectified

linear unit (ReLU) activation function in this paper. W_{ji}^l and b_j^l are the filters and biases to be trained for this layer.

Each convolutional layer is followed by a pooling layer. The pooling layer can be regarded as a subsampling or compulsory blurring operation. It is designed to reduce the resolution of the feature maps. Another side effect of the pooling layer is to keep shifting, rotational, and scaling invariance of the features. Max pooling is employed in this paper.

The fully connected layers further embed the features generated from convolutional and pooling layers into a feature space with better separation for different classes. The input of the fully connected layer is a vector $x^{(l-1)}$ whose dimension is m_f . The output of the fully connected layer is shown as

$$\alpha_j^l = \sigma \left(\sum_{i=1}^{m_f} x_i^{(l-1)} * W_{ji}^l + b_j^l \right) \quad (5)$$

where $x_i^{(l-1)}$ is the i th element of $x^{(l-1)}$ and α_j^l is the j th output unit of the l th fully connected layer. $\sigma(\cdot)$ is the activation function, which is taken as ReLU activation function as well. The filters W_{ji}^l and biases b_j^l constitute the trainable parameters for the fully connected layers. Batch normalization layers are inserted after each of the convolutional layers.

Softmax function is used as the final output layer to solve the multiclass classification problem. Let α_i be the output of the fully connected layers, and W_s be the weight matrices of softmax function; the output of the softmax function is

$$p(y_i = j | \alpha_i; W_s) = \frac{e^{W_{sj}^T \alpha_i}}{\sum_{k=1}^K e^{W_{sk}^T \alpha_i}} \quad (6)$$

where K is the number of classes, and the output exactly interprets the probability of pixel i belonging to the class j .

To simplify the notation, we denote parameter Θ as $\{(W^l, b^l)_{l=1,3,5,6}, W_s\}$, which is the combination of the filters, weight matrices, and biases of the whole CNN. Then, the energy function of the CNN term can be written as

$$E_c(F_\Theta | X, Y) = - \sum_{i=1}^N \sum_{j=1}^K 1\{y_i = j\} \log P(y_i = j | F_\Theta) \quad (7)$$

where $P(y_i = j | F_\Theta)$ denotes the probability of predicting x_i to have class label j .

3) *Pairwise Smoothness Term*: The pairwise smoothness term $E_p(Y | X)$ is defined to enforce that the neighboring pixels should have similar labels [50], which is defined as

$$E_p(Y | X) = \alpha_p \sum_{i=1}^N \sum_{j \in \mathcal{N}(i)} S_{ij} \quad (8)$$

where α_p is the pairwise smoothness factor and $\mathcal{N}(i)$ is the neighboring pixel set of pixel i . S_{ij} is defined as

$$S_{ij} = |y_i - y_j| \exp \left(- \frac{\|v_i - v_j\|_2^2}{2\sigma_m} \right) \quad (9)$$

where v_i is a feature vector at pixel i , which should be taken as features significantly changing values across the edges in the image. Pauli matrix components are employed in this paper. σ_m is the mean squared distance between features of

adjacent pixels i and j . Pairwise smoothness term strongly enforces that adjacent pixels should take the same label, and encourages the class label boundaries to align with edges in the image. If pixels i and j are located within flat regions, $\exp(-(\|v_i - v_j\|_2^2)/2\sigma_m)$ in (9) is large; then, minimizing S_{ij} strongly enforces that labels y_i and y_j should have the same value. But if the neighboring pixels are located across strong edges, $\exp(-(\|v_i - v_j\|_2^2)/2\sigma_m)$ is smaller, thus the difference between labels of neighboring pixels i and j is acceptable during optimization process [50].

4) *Discussion on the Final Energy Function*: Summarized from the above-discussed formulations, the final integrated energy function can be written as

$$\begin{aligned} E(Y, F_\Theta | X, L) &= \underbrace{\sum_{i=1}^N \psi_i^{\text{semi}}(y_i | X, L)}_{\text{semisupervision term}} \\ &\quad - \underbrace{\sum_{i=1}^N \sum_{j=1}^K 1\{y_i = j\} \log P(y_i = j | F_\Theta)}_{\text{CNN term}} \\ &\quad + \underbrace{\alpha_p \sum_{i=1}^N \sum_{j \in \mathcal{N}(i)} |y_i - y_j| \exp \left(- \frac{\|v_i - v_j\|_2^2}{2\sigma_m} \right)}_{\text{Pairwise smoothness term}}. \end{aligned} \quad (10)$$

The energy function is the combination of semisupervision term, CNN term, and pairwise smoothness term. Semisupervision term is to constrain the human-labeled pixels to be fixed with their labeled categories. CNN term is a key term to learn discriminative features for predicting the pixelwise class labels with the guidance of fixed partial human-labeled pixels and iteratively estimated class labels on unlabeled pixels. The pairwise smoothness term is to enforce the smoothness of the estimated class label map and alignment of class label boundaries with image edges.

In the energy function, the unknown variables include the pixelwise class labels Y (discrete variables) and CNN parameters Θ (continuous variables) in CNN F_Θ .

D. Optimization

To solve the unknown variables in (10), we design an iterative algorithm to optimize the energy function. The optimization problem can be decomposed into two subproblems, i.e., *CNN learning-subproblem* which aims to learn the network parameters Θ in CNN F_Θ with fixed class labels Y and *label propagation-subproblem* which aims to update Y with fixed CNN F_Θ . These two subproblems are dependent on each other. The CNN is supervised by the propagated class labels and, in turn, provides class label predictions for the class label propagation. The dependence between the two subproblems is realized in an alternating optimization solution. In each iteration, the optimization of the CNN parameters is first implemented by learning a deep neural network for class label prediction, and then, the optimization of the class labels is conducted through graph-based class label propagation.

1) *CNN Learning-Subproblem*: This subproblem dedicates to learn a CNN based on the current class labels Y , which keep constant in this subproblem. For iteration $t = 1$, the current class labels Y are taken as the K -Wishart or superpixel initialized class labels (as will be discussed in Section III-E), and for iteration $t > 1$, they are taken as the class labels estimated in iteration $t - 1$.

In CNN learning-subproblem, since class labels Y are taken as fixed variables, the energy function in (10) can be rewritten as

$$E(F_\Theta|X, Y) = - \sum_{i=1}^N \sum_{j=1}^K 1\{y_i = j\} \log P(y_i = j|F_\Theta). \quad (11)$$

Network parameters Θ in CNN F_Θ can be optimized by minimizing the energy function in (11). In a CNN, the convolutional layer, the pooling layer, the fully connected layer, and the softmax classifier act as processes of the forward propagation, which is used for predicting the class labels of the input feature patches. The backward propagation [53] is the process for learning the network parameters Θ in F_Θ . The optimization of the energy function in (11) is conducted using the stochastic gradient descent algorithm. In the t th iteration, the parameters are updated by

$$\Theta_{t+1} = \Theta_t - \tau \frac{\partial E(F_\Theta|X, Y)}{\partial \Theta} \quad (12)$$

where τ is the learning rate. With the updated CNN parameters Θ , the function defined in (6) exactly gives the probabilities of a pixel belonging to different classes. The open-source implementation MatConvNet¹ is used to train the CNN.

2) *Label Propagation-Subproblem*: Based on the updated CNN F_Θ in CNN learning (CL)-subproblem, this subproblem aims to estimate pixelwise class labels Y . The pixels are labeled by the retrained CNN F_Θ and then smoothed by the pairwise smoothness constraint. During the class label propagation, human-labeled pixels are always enforced by the semisupervision term. Since the CNN is a fixed network in this subproblem, the energy function in (10) can be rewritten as

$$\begin{aligned} E(Y|X, L, F_\Theta) &= \sum_{i=1}^N \psi_i^{\text{semi}}(y_i|X, L) \sum_{i=1}^N \sum_{j=1}^K 1\{y_i = j\} \log P(y_i = j|F_\Theta) \\ &\quad + \alpha_p \sum_{i=1}^N \sum_{j \in \mathcal{N}(i)} |y_i - y_j| \exp\left(-\frac{\|v_i - v_j\|_2^2}{2\sigma_m}\right). \end{aligned} \quad (13)$$

This class label assignment problem is a combinatorial optimization problem, which can be regarded as a standard MRF model [50]. The class labels Y over the graph constitute an MRF, and (13) is the energy function on it. The three terms of the energy function enforce class label constraints from human-labeled pixels, agreement between the CNN predictions and target class labels, and class label smoothness, respectively. The minimization of these three terms enforces that the predicted class labels should be smooth and consistent with the human-labels and CNN predictions.

Labeling problem in an MRF is an NP-hard problem. However, the optimal solution can be approximately achieved using the graph cut [51] or belief propagation [54] algorithm. The belief propagation algorithm adopts “up-down-left-right” message passing schedule, which makes it converge fast [50]. Therefore, we employ belief propagation algorithm [54] in this paper.

E. Initialization

Our proposed classification model aims at improving the CA using limited numbers of labeled pixels. However, it is difficult to learn a discriminative classifier training only the labeled pixels. For semisupervised classification methods, incorporating class information of reliable unlabeled pixels in the classification process is an effective way to promote the classification performance [27], [52]. Therefore, we design two initialization methods according to different similarity measurements in this paper. The generated class labels act as input for our optimization algorithm as described in Section III-D.

1) *K-Wishart Initialization*: In this section, we first briefly introduce the K -Wishart polarimetric statistical distribution theory and then introduce our proposed pixelwise K -Wishart initialization method.

The development of PolSAR data statistical distribution theory follows the tendency from single-look to multilook and from homogeneity to heterogeneity. The complex Wishart distribution which is based on the Gaussian hypothesis was first proposed in [49] and has been extensively used in many PolSAR image classification methods [1], [9], [10], [19]. However, along with the research on polarimetric statistical distribution theory, it is discovered that the complex Wishart distribution is suitable for describing homogeneous areas, but does not work well in heterogeneous areas. To address this problem, Douglgeris *et al.* [16] presented multilook K -Wishart distribution, which is based on more general non-Gaussian hypothesis. It has been demonstrated that the K -Wishart distribution fits for both homogeneous and heterogeneous areas, and exhibits improved polarimetric distinction.

Based on the K -Wishart distribution, a K -Wishart distance is defined to reveal the similarity of a pixel to a certain terrain type, which is defined as

$$\begin{aligned} d_m(T) &= M \ln |V_m| + \ln(\Gamma(\lambda)) - \frac{\lambda + Md}{2} \ln(M\lambda) \\ &\quad - \frac{\lambda - Md}{2} \ln(\text{Tr}(V_m^{-1}T)) \\ &\quad - \ln B_{\lambda-Md}(2\sqrt{M\lambda\text{Tr}(V_m^{-1}T)}) \end{aligned} \quad (14)$$

where T is the pixelwise coherence matrix and V_m is the cluster center of mean coherency matrix of class m . $\text{Tr}(\cdot)$ and $|\cdot|$ denote the trace operation and the determinant, respectively. $B_\rho(\cdot)$ is the modified Bessel function of the second kind with order ρ , $\Gamma(\cdot)$ is the standard Gamma function, M represents the number of looks, d is the vector dimension, and λ is the shape parameter representing the data distribution.

Based on the defined K -Wishart distance, we design a semi-supervised K -Wishart initialization method. Different from the unsupervised K -Wishart PolSAR image classification method

¹[Online]. Available: <http://www.vlfeat.org/matconvnet/>

in [16] and [17], our proposed method utilizes the given labeled pixels as supervision and propagates class information from the labeled pixels to the unlabeled ones. We first use the given small proportion of pixels with ground-truth class labels to determine the initial cluster centers, i.e., take the mean value of coherency matrices T of labeled pixels in each class as the cluster center. Then, we run the pixelwise K -Wishart clustering algorithm by iteratively refining cluster centers and class label assignment, generating a full image initialized class label map for consequent class label optimization. We denote our proposed method with K -Wishart initialization as KWGraphCNN hereinafter.

2) *Supapixel Initialization*: It is extensively recognized that class labels of PolSAR images are spatially consistent, which means that neighboring pixels tend to fall into the same terrain categories [17], [19]. According to this heuristic prior, PolSAR images can be segmented into small homogeneous regions, i.e., superpixels where pixels inside share the same class labels. In this case, if a superpixel overlaps with a labeled pixel, other pixels inside the homogeneous region can be initialized with the same class label. Since the available labeled pixels are very limited, this is a promising way to incorporate more class information.

In accordance with the above-mentioned analysis, we design an initialization method based on the superpixel segmentation. We first perform simple linear iterative cluster (SLIC) [56] superpixel segmentation² on PauliRGB images of PolSAR data. Then, class labels propagate from the given labeled pixels to the homogeneous unlabeled ones according to the superpixel segmentation result. For each superpixel, if it includes a labeled pixel, all pixels within it are initialized with the same class label as the labeled pixel and added to the training pixel set. Finally, an augmented training pixel set is produced, acting as input for the optimization algorithm in Section III-D. The proposed method with superpixel initialization is denoted as SPGraphCNN in Sections IV and V.

F. Summary of the Algorithm

Algorithm 1 illustrates the pipeline of the proposed algorithm. We first conduct speckle noise reduction utilizing the refined Lee filter [57]. And then, we extract raw features from the PolSAR data (see Section III-B) and generate an initialized class label map based on the K -Wishart distribution hypothesis or superpixel segmentation of PauliRGB images (see Section III-E). Finally, we iteratively and alternately update class labels and CNN parameters by optimizing our designed graph-based semisupervised deep learning model (see Sections III-C and III-D). The output of the proposed algorithm includes the class labels Y and CNN F_{Θ} .

IV. EXPERIMENTS

In this section, we evaluate the proposed method quantitatively and qualitatively on two real benchmark PolSAR images. We first introduce the experimental data and parameter settings in Section IV-A. Then, we conduct ablation study in Section IV-B, justifying the effectiveness of the

Algorithm 1 Graph-Based Semisupervised Deep Learning Model for PolSAR Image Classification

Input: pixelwise coherence matrix T , learning rate τ , momentum parameter, pairwise smoothness parameter α_p
Output: pixelwise class labels Y , CNN F_{Θ}
1: Refined Lee speckle denoising
2: Raw Feature Extraction and Initialization
3: **While** not meet the termination criterion **do**
4: Solve *CNN learning-subproblem* to update CNN parameters Θ
5: Solve *Label propagation-subproblem* to update class labels Y
6: **end**

four key components of the proposed model: initialization method, CNN learning, label propagation, and iterative optimization. We demonstrate the effectiveness of the proposed method by comparison with other nine state-of-the-art methods in Section IV-C. The compared methods include SVM [5], WBRBM [9], WDBN [9], WDSN [10], CNN [11], subspace multinomial logistic regression with multilevel logistic prior (MLR_{sub}MLL) [58], Wishart classifier [1], semisupervised spatial anchor (SSA) [27], and semisupervised feature extraction with neighborhood constraints (SNCs) [29], where SVM, WBRBM, WDBN, WDSN, CNN, and MLR_{sub}MLL are supervised methods, and Wishart classifier, SSA, and SNC are semisupervised methods. Finally, taking Flevoland area data for example, we analyze the influence of the parameter settings on the proposed methods in Section IV-D.

A. Experimental Data and Parameter Setting

Fig. 3 shows the employed experimental PolSAR images. The first image is for Flevoland area in The Netherlands, which was acquired by NASA/JPL AIRSAR on August 16, 1989. The size of the image is 750×1024 . The PauliRGB image of Flevoland area data is shown in Fig. 3(a). The ground-truth class labels and the corresponding color codes are shown in Fig. 3(b). There are 15 classes in the image, including *water*, *barley*, *peas*, *stembean*, *beet*, *forest*, *bare soil*, *grass*, *rapeseed*, *lucerne*, *wheat 1*, *wheat 2*, *building*, *potato*, and *wheat 3*. The second data are a four-look NASA/JPL AIRSAR L-band data of San Francisco area. The size of the image is 900×1024 . The PauliRGB image of San Francisco area data is shown in Fig. 3(c). Ground-truth class labels of this data are unavailable. We adopt the same class label set as [11], where the image is classified into five classes: *vegetation*, *water*, *low-density urban*, *high-density urban*, and *slanted urban*. The labeled pixels and validation pixels are marked in the image.

We set the network training parameters as follows. The size of the pixelwise input feature patch fed into the CNN is $8 \times 8 \times 6$. We use 20 kernels in the first convolutional layer and 50 kernels in the second one. The kernel sizes of the first and second convolutional layers are set as 3×3 and 2×2 , respectively. The pooling size of the two max-pooling layers is taken as 2×2 , and the stride is set as 2. The first fully connected layer contains 500 units, and the unit

²[Online]. Available: <https://ivrl.epfl.ch/research/superpixels>

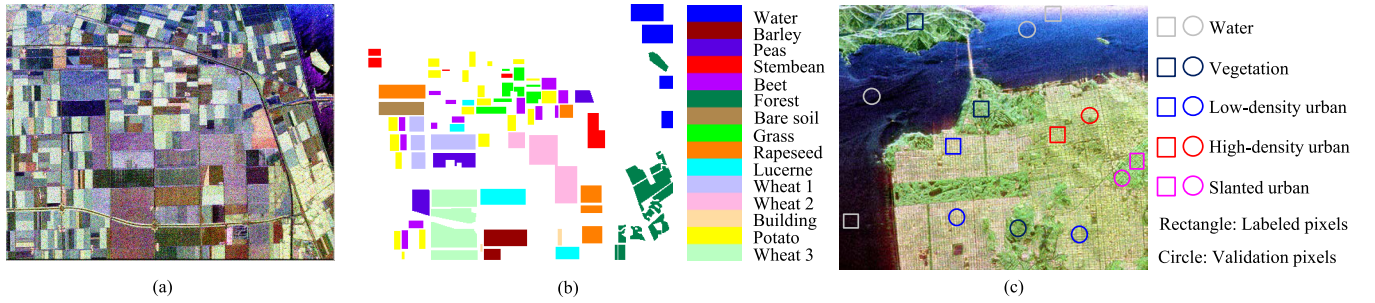


Fig. 3. Original images of Flevoland and San Francisco areas. (a) PauliRGB image of Flevoland area. (b) Ground-truth class labels and color codes of Flevoland area data. (c) PauliRGB image of San Francisco area.

number of the second one takes the value of the class number. For the optimization algorithm, the learning rate τ and the momentum parameter are set to 0.001 and 0.9, respectively. The batch size is set as 100. The CNN parameters Θ to be learned are initialized with random values in the first iteration. For the following iterations, we inherit the CNN parameters of the previous iteration for network fine-tuning. For KWGraphCNN and SPGraphCNN methods, the epoch numbers of the first iteration are set as 30 and 60, respectively, and the epoch numbers of the following iterations are both set as 10. Moreover, the pairwise smoothness parameter α_p is set as 5, and 1% pixels with ground-truth class labels are taken as input for our proposed method. We will evaluate the effect of parameter settings on classification performance in Section IV-D. All the experiments are implemented in the MATLAB environment on a Dell T7910 workstation with GeForce GTX TITAN GPU and 64-GB memory.

For convenience, we define the CA of a class as the ratio of the number of pixels correctly classified for the class to the total number of pixels in this class. The *overall CA* is defined as the ratio of the number of correctly classified pixels in the whole image to the total number of pixels in the image.

B. Ablation Study

To evaluate the effectiveness of the four key components of the proposed model: initialization method, CL, LP, and iterative optimization, we conduct three groups of experiments as follows:

- 1) using 1% labeled pixels without any initialization (denoted as GraphCNN);
- 2) using 1% labeled pixels and K -Wishart initialization (KWGraphCNN);
- 3) using 1% labeled pixels and superpixel initialization (SPGraphCNN).

In each group of experiments, we present results in four different cases: the initialized class label map, classification result after one round of CL, classification result after one round of CL + LP, and classification result after three rounds of CL + LP. Fig. 4 shows all the classification results, and Table II lists the numerical results.

The experimental results of the GraphCNN classifier are presented in Fig. 4(a1)–(a4). The input class labels for GraphCNN are displayed in Fig. 4(a1). Fig. 4(a2) exhibits the classification result after performing CNN learning, where

a great deal of misclassifications can be observed. In the region marked by black oval, most of the *rapeseed* pixels are misclassified as *wheat 1* class. In addition, pixels which should be classified as *barley* (marked by red rectangle) are wrongly assigned with *grass* class labels.³ The reason accounting for these misclassifications is that the available labeled pixels are too limited, making that the learned network fails to distinguish these terrains. Analyzed from the above-mentioned analysis, we find that the classification performance of the classifier using 1% labeled pixels without any initialization is unsatisfactory.

Fig. 4(b1)–(b4) and (c1)–(c4) shows the classification results of our proposed method based on the K -Wishart and superpixel initialized class label maps, respectively. Fig. 4(b1) shows the K -Wishart initialized class label map, while Fig. 4(c1) displays the superpixel initialized class label map. Fig. 4(b2) presents the classification result after performing CNN learning and prediction on K -Wishart initialization. Compared with the K -Wishart initialized class label map, many previously misclassified pixels are assigned with correct class labels, which exactly illustrates the discriminative feature learning and classification capacity of CNN. The *Overall CA* value of Fig. 4(b2) is 5.63% higher than that of Fig. 4(b1). Similarly, as shown in Fig. 4(c2), the classification result after performing CNN optimization on superpixel initialization is in favorable agreement with the ground-truth class labels. Compared with the classification results in Fig. 4(b2) and (c2), the class label maps in Fig. 4(b3) and (c3) present better spatial consistency with clearer edges and less isolated pixels, which illustrates the class label smoothing and boundary alignment effects of the pairwise smoothness constraint. The *Overall CA* values of Fig. 4(b3) and (c3) are 5.51% and 1.90% higher than those of Fig. 4(b2) and (c2), respectively. To demonstrate the effectiveness of the iterative algorithm, we conduct three rounds of CL + LP optimization on the Flevoland area data. It is noteworthy that the classification results after three rounds of CL + LP are in better conformity with the ground-truth class labels and exhibit better contextual consistency [refer to regions marked by black ovals in Fig. (b4) and (c4)]. The *Overall CA* values of Fig. 4(b4) and (c4) are increased by 1.75% and 1.70% compared with Fig. 4(b3) and (c3), respec-

³The ground-truth class labels of this area are not provided in Fig. 3(b). We obtain the class information from data resources [19], [50].

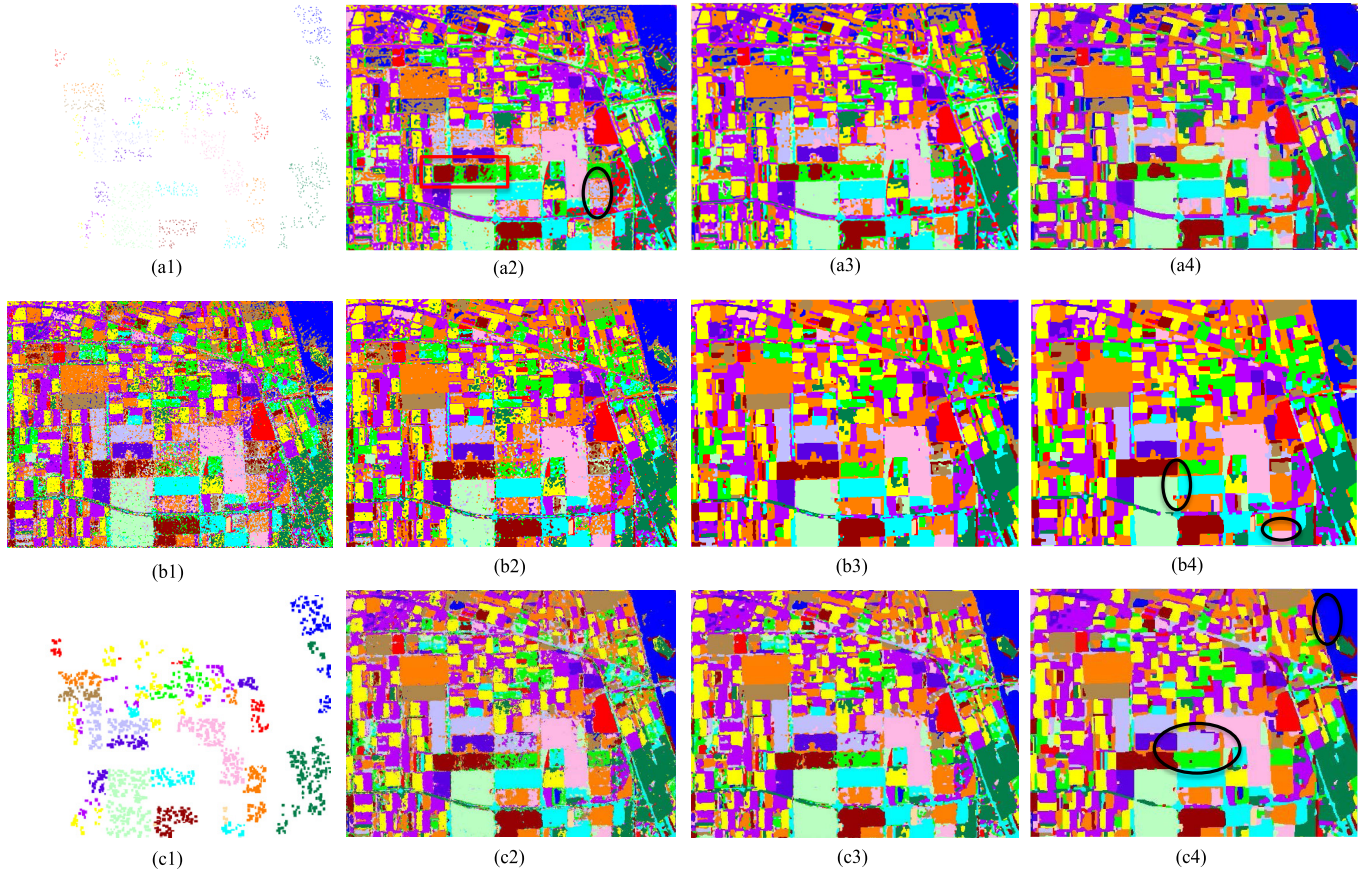


Fig. 4. Classification results of Flevoland area data for the proposed method. (a1) Initialized class label map in GraphCNN group. (a2) Classification result after one round of CL in GraphCNN group. (a3) Classification result after one round of CL + LP in GraphCNN group. (a4) Classification result after three rounds of CL + LP in GraphCNN group. (b1) Initialized class label map in KWGraphCNN group. (b2) Classification result after one round of CL in KWGraphCNN group. (b3) Classification result after one round of CL + LP in KWGraphCNN group. (b4) Classification result after three rounds of CL + LP based in KWGraphCNN group. (c1) Initialized class label map in SPGraphCNN group. (c2) Classification result after one round of CL in SPGraphCNN group. (c3) Classification result after one round of CL + LP in SPGraphCNN group. (c4) Classification result after three rounds of CL + LP in SPGraphCNN group.

tively. We can observe from Table II that the final *Overall CA* values of KWGraphCNN and SPGraphCNN are 9.41% and 10.48% higher than GraphCNN. Moreover, KWGraphCNN and SPGraphCNN behave better than GraphCNN on areas without ground-truth class labels [see the area marked by red rectangle in Fig. 4(a2)], which demonstrates the strong generalization ability of our proposed methods. Analyzed from the above-discussed experimental results, we justify that the initialization method, CNN learning, label propagation, and iterative optimization can effectively promote the classification performance.

C. Results and Comparisons

1) *Flevoland Area Data*: To demonstrate the performance of our proposed model, we compare KWGraphCNN and SPGraphCNN methods with other nine state-of-the-art methods on the Flevoland area data. Fig. 5 shows the classification results using these 11 methods. Table III reports the CA values of them. The class label information of the Flevoland area data is shown in Fig. 3(b).

Fig. 5(a) shows the classification result using the SVM classifier. We can observe from Fig. 5(a) that the

misclassification is severe in the whole image, especially in the top and left part of the image. This indicates that the SVM classifier with handcrafted features does not behave well on PolSAR images with complex terrain classes. Fig. 5(b) and (d) exhibits classification results using the WBRBM classifier and the WDSN classifier, respectively. Lots of misclassified pixels can be observed in Fig. 5(b) and (d). For example, some *rapeseed* pixels are classified as *wheat 1* class and vice versa [marked by black ovals in Fig. 5(b) and (d)]. We can also see some internal disconnections in the image [marked by the black rectangle in Fig. 5(b)]. Classification result using CNN is shown in Fig. 5(e). The whole class label map is much clearer than the previously analyzed ones. However, the *grass* class is overinfluencing the whole image. One instance is that the long narrow part [marked by the black rectangle in Fig. 5(e)] which belongs to *lucerne* is misclassified as *grass* instead. Fig. 5(f) shows the classification result using the MLRsubMLL classifier. It can be observed from Fig. 5(f) that *wheat 1* pixels are partly misclassified as *rapeseed* and partly wrongly assigned with *wheat 3* class labels [marked by black oval in Fig. 5(f)]. In addition, most of the *grass* pixels are misclassified either as *beet* class [marked by black rectangle in Fig. 5(f)] or *wheat 3* class [marked by red rectangle in Fig. 5(f)].

TABLE II
CA VALUES (%) OF FLEVLAND AREA DATA FOR THE PROPOSED METHOD

Group	Object	Water	Barley	Peas	Stembean	Beet	Forest	Bare soil	Grass
GraphCNN	1 round of CL	91.27	94.02	91.04	95.60	91.31	96.71	81.38	70.82
	1 round of CL+LP	91.22	92.99	90.77	93.84	93.06	95.41	85.83	70.41
	3 rounds of CL+LP	94.60	94.92	93.34	94.46	95.95	96.14	83.23	70.82
KWGraphCNN	Initialization	91.43	89.10	94.11	94.61	91.63	84.23	94.82	75.09
	1 round of CL	92.85	95.19	98.06	97.65	97.06	95.49	99.43	81.95
	1 round of CL+LP	98.91	97.81	99.91	98.70	99.60	99.54	100.00	91.23
	3 rounds of CL+LP	98.69	98.08	99.96	98.08	99.36	99.57	100.00	92.50
SPGraphCNN	Initialization	100.00	98.79	95.60	100.00	89.21	100.00	96.78	93.36
	1 round of CL	99.67	97.60	95.88	98.33	98.47	97.04	99.27	67.36
	1 round of CL+LP	100.00	98.45	98.36	98.82	98.55	98.54	100.00	75.20
	3 rounds of CL+LP	100.00	99.68	99.12	99.07	99.76	98.81	100.00	90.72
Group	Object	RapeSeed	Lucerne	Wheat	Wheat 2	Building	Potato	Wheat 3	Overall CA
GraphCNN	1 round of CL	69.52	95.43	70.39	94.79	65.54	88.41	94.77	84.43
	1 round of CL+LP	69.19	93.85	75.71	93.77	88.19	89.24	93.73	87.47
	3 rounds of CL+LP	67.03	94.17	76.46	95.69	61.97	89.41	94.38	88.34
KWGraphCNN	Initialization	76.14	95.11	72.90	81.21	75.48	86.14	81.11	84.86
	1 round of CL	83.96	98.30	85.81	83.78	71.19	87.41	86.61	90.49
	1 round of CL+LP	96.87	98.10	96.81	86.10	73.45	95.27	93.89	96.00
	3 rounds of CL+LP	99.15	98.10	99.71	85.86	76.84	99.63	99.01	97.75
SPGraphCNN	Initialization	95.05	100.00	98.36	100.00	100.00	86.63	98.48	98.03
	1 round of CL	96.62	98.91	95.20	97.58	96.05	95.84	98.26	95.22
	1 round of CL+LP	98.21	99.51	96.45	98.82	97.25	97.51	99.05	97.12
	3 rounds of CL+LP	99.26	99.72	99.10	99.65	98.31	98.83	99.50	98.82

Classification result using the Wishart classifier is shown in Fig. 5(g). The confusion between *bare soil* and *water* classes is severe [marked by the red rectangle in the top-right part of Fig. 5(g)] in this image, because *bare soil* and *water* both belong to surface scattering mechanism, and the Wishart classifier fails to distinguish them. Moreover, some *rapeseed* and *wheat 1* pixels are confused [marked by black ovals in Fig. 5(g)].

Fig. 5(h) and (i) gives the classification results using the SSA classifier and the SNC classifier, respectively. The original papers [27] and [29] did not provide the classification results of the whole image but gave the classification results of the ground-truth class label masked pixels instead. We can find from Fig. 5(h) and (i) that the misclassifications between groups of *grass* and *forest* and *rapeseed* and *wheat 1* are severe [marked by black rectangles and ovals in Fig. 5(h) and (i)]. Fig. 5(c) displays the classification result using the WDBN classifier. Fig. 5(j) and (k) shows the classification results of our proposed methods using 1% pixels with ground-truth class labels. We can find improved performance from Fig. 5(c), (j), and (k), which exhibit better agreement with the ground-truth class labels and more favorable spatial connectivity.

The numerical results of the 11 methods are listed in Table III. The third column in Table III denotes the percentages of the used ground-truth class labels. The first six methods are supervised methods, where SVM [5], WBRBM [9], WDBN [9], WDSN [10], and MLR_{sub}MLL [58] use 5% pixels with ground-truth class labels, and CNN [11] uses 10% pixels with ground-truth class labels in training. Wishart classifier [1], SSA [27], and SNC [29] are semisupervised

methods, where 1% pixels with ground-truth class labels are utilized. We illustrate the CA values of our proposed methods using 1% and 5% pixels with ground-truth class labels in Table III.

To fairly compare the classification results, we use the classification results of our proposed methods utilizing 5% pixels with ground-truth class labels to compare with the supervised methods, and the classification results utilizing 1% pixels with ground-truth class labels to compare with the semi-supervised methods. From Table III, we can find that: 1) the KWGraphCNN classifier using 5% pixels with ground-truth class labels outperforms the six supervised methods (SVM, WBRBM, WDBN, WDSN, CNN, and MLR_{sub}MLL) by 12.45%, 12.80%, 0.75%, 5.64%, 5.86%, and 19.93%, respectively, while the SPGraphCNN classifier using 5% pixels with ground-truth class labels outperforms them by 13.30%, 13.65%, 1.60%, 6.49%, 6.71%, and 20.78% and 2) the KWGraphCNN classifier using 1% pixels with ground-truth class labels outperforms other three semisupervised methods (Wishart classifier, SSA, and SNC) by 18.93%, 10.17%, and 13.11%, respectively, while SPGraphCNN classifier using 1% pixels with ground-truth class labels outperforms them by 20.00%, 11.24%, and 14.18%. Moreover, our proposed methods using 1% pixels with ground-truth class labels still outperform other supervised methods using 5% or 10% pixels with ground-truth class labels. The above-mentioned comparison results demonstrate the strong discriminative ability of our proposed methods to distinguish different terrain classes using a very limited number of labeled pixels.

Although our proposed methods outperform other compared methods in terms of numerical results, some small objects are

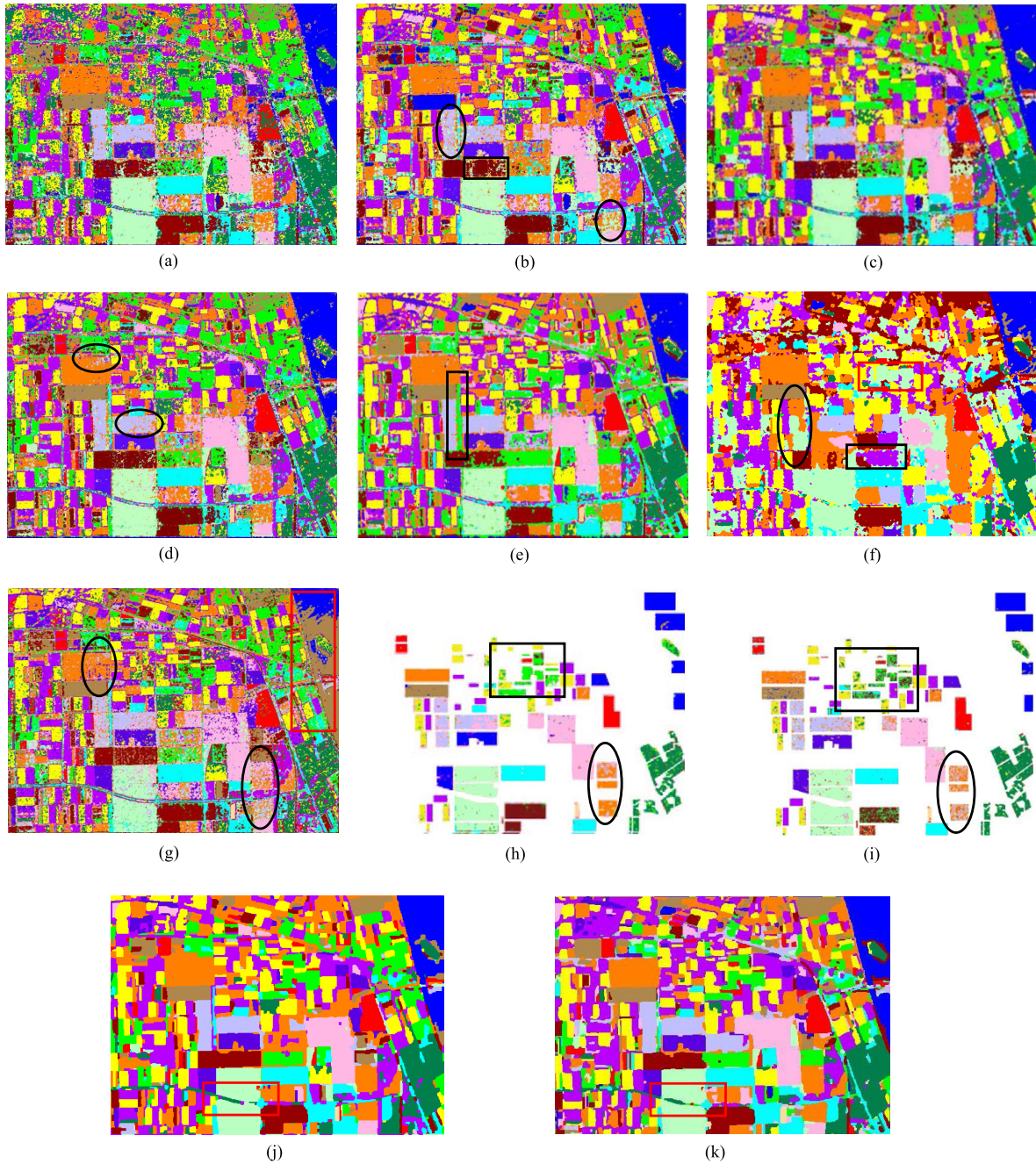


Fig. 5. Classification results of Flevoland area data with different methods. (a) SVM classifier [5]. (b) WBRBM classifier [9]. (c) WDBN classifier [9]. (d) WDSN classifier [10]. (e) CNN classifier [11]. (f) MLRsubMLL classifier [58]. (g) Wishart classifier [1]. (h) SSA classifier [27]. (i) SNC classifier [29]. (j) KWGraphCNN classifier. (k) SPGraphCNN classifier.

over-smoothed and misclassified after iterations of optimization. For example, the road area [marked by red rectangles in Fig. 5(j) and (k)] which should be long and continuous is cut to several disconnected segments for our proposed methods. This is because these small object areas are mixed with pixels of different terrain classes, or the edges of them are weak and unclear. Therefore, the small objects shrink or even disappear after rounds of successive MRF smoothing.

The time complexity of our proposed model depends on the time complexities of the two tasks conducted in the

optimization process, i.e., CNN learning and label propagation. The time complexity of CNN learning is determined by the number of convolutional layers, the filter number in each layer, and the input and output feature channel numbers of each layer in the CNN, which can be referred to [59]. The belief propagation algorithm is employed to solve the label propagation problem in this paper, which has a linear time complexity [54]. The total running time of the KWGraphCNN method for the Flevoland area data is 14 min 35 s, including the network training (13 min 38 s) and label propagation (57 s).

TABLE III
CA VALUES (%) OF FLEVOLAND AREA DATA WITH DIFFERENT METHODS

Method	Supervision	Used labels	Water	Barley	Peas	Stembean	Beet	Forest	Bare soil	Grass
SVM [5]	Supervised	5%	97.82	97.17	78.94	63.55	90.91	93.82	96.58	87.59
WBRBM [9]	Supervised	5%	99.71	98.16	96.46	91.30	94.44	89.71	0.14	36.97
WDBN [9]	Supervised	5%	99.90	99.49	98.68	96.71	98.14	96.45	97.34	95.39
WDSN [10]	Supervised	5%	98.88	97.05	96.81	96.00	96.22	90.85	99.26	90.13
CNN [11]	Supervised	10%	98.88	96.90	88.89	92.58	89.74	93.95	99.94	79.20
MLRsubMLL [58]	Supervised	5%	92.64	80.88	10.97	84.88	82.74	97.06	99.84	5.36
Wishart [1]	Semi-supervised	1%	50.39	95.88	92.13	90.49	82.30	81.20	99.22	61.77
SSA [27]	Semi-supervised	1%	92.63	96.23	93.01	94.14	94.17	86.44	96.19	65.25
SNC [29]	Semi-supervised	1%	96.79	64.45	91.83	92.35	96.66	90.44	92.11	50.54
KWGraphCNN	Semi-supervised	1%	98.69	98.08	99.96	98.08	99.36	99.57	100.00	92.50
SPGraphCNN	Semi-supervised	1%	100.00	99.20	99.62	98.57	99.76	99.87	100.00	95.21
KWGraphCNN	Semi-supervised	5%	100.00	97.44	99.62	98.76	99.60	99.93	100.00	98.35
SPGraphCNN	Semi-supervised	5%	100.00	99.95	99.83	100.00	99.92	94.68	100.00	97.29
Method	Supervision	Used labels	RapeSeed	Lucerne	Wheat	Wheat 2	Building	Potato	Wheat 3	Overall CA
SVM [5]	Supervised	5%	81.28	96.04	90.79	81.15	0.43	53.97	95.48	85.87
WBRBM [9]	Supervised	5%	72.48	98.49	91.67	69.31	76.68	89.92	96.99	85.52
WDBN [9]	Supervised	5%	95.90	98.47	97.67	94.79	88.56	98.08	98.55	97.57
WDSN [10]	Supervised	5%	86.18	96.05	93.22	81.20	84.12	87.63	95.79	92.68
CNN [11]	Supervised	10%	93.10	92.21	93.62	91.82	87.18	87.24	94.46	92.46
MLRsubMLL [58]	Supervised	5%	97.41	96.69	40.90	68.87	89.83	97.56	94.64	78.39
Wishart [1]	Semi-supervised	1%	55.70	92.38	69.11	89.04	84.35	84.91	79.06	78.82
SSA [27]	Semi-supervised	1%	80.88	90.19	90.74	76.96	78.25	87.71	90.98	87.58
SNC [29]	Semi-supervised	1%	69.19	92.78	83.83	71.62	73.63	85.53	89.33	84.64
KWGraphCNN	Semi-supervised	1%	99.15	98.10	99.71	85.86	76.84	99.63	99.01	97.75
SPGraphCNN	Semi-supervised	1%	99.26	99.72	99.10	99.65	98.31	98.83	99.50	98.82
KWGraphCNN	Semi-supervised	5%	98.21	97.66	99.82	92.52	85.14	99.83	97.93	98.32
SPGraphCNN	Semi-supervised	5%	99.49	99.92	98.71	99.98	100.00	99.98	99.51	99.17

The total running time of the SPGraphCNN method for the Flevoland area data is 10 min 42 s, including the network training (9 min 45 s) and label propagation (57 s).

2) *San Francisco Area Data*: Fig. 6 shows the classification results of the San Francisco area data using the proposed method, SVM classifier [5], Wishart classifier [1], CNN [11] classifier, and MLRsubMLL [58] classifier. Table IV reports the CA values of these six methods. Other classifiers used for the Flevoland area data are not compared because the original papers did not apply them on these data.

Fig. 6(a) gives the classification result using the SVM classifier. We can see from Fig. 6(a) that the *high-density urban*, *low-density urban*, and *slanted urban* classes are severely mixed. The *slanted urban* area which locates at the right middle part of the image can hardly be discriminated. This is mainly due to the weak representative ability of the handcrafted features. Fig. 6(b) shows the classification result using the Wishart classifier. We can see class confusion between *high-density urban* and *low-density urban*, and *vegetation* and *water* from Fig. 6(b). This is because the employed Wishart distance in this method fails to distinguish two similar scattering mechanisms. The classification result using MLRsubMLL is shown in Fig. 6(c), where the misclassification of *low-density urban* and *slanted urban* terrains can be observed obviously. Fig. 6(d)–(f) shows the classification result using CNN, KWGraphCNN, and SPGraphCNN classifiers, respectively. Comparing the results of the CNN classifier and our methods, we can find that: 1) the classification results

using our methods present better contextual consistency than the CNN classifier and 2) for the right bottom part of the image, CNN classifier assigns *high-density urban* class to most of the pixels in this area, while our methods classify them as *low-density urban* class. After referring to the C-band PolSAR image of this area [17], it can be discovered that it is more reasonable to classify this area as *low-density urban* class. The *Overall CA* value of the KWGraphCNN classifier is 42.88%, 31.53%, 8.40%, and 23.63% higher than other four methods (SVM, Wishart classifier, CNN, and MLRsubMLL), while the SPGraphCNN classifier outperforms them by 41.79%, 30.44%, 7.31%, and 22.54%, respectively.

D. Analysis of Parameter Settings

We next analyze the effects of different parameters on the classification performance. From Sections IV-D1–IV-D5, we study the influence of parameters based on the KWGraphCNN method. From Section IV-D6, we study the effect of superpixel size specific to the SPGraphCNN method.

1) *Pairwise Smoothness Parameter and Iteration Number*: For the proposed method, α_p and iteration number are two fundamental parameters, which greatly influence the final result. α_p determines the tightness of class label smoothness constraint. Larger α_p denotes that the pairwise smoothness term plays a more important role during the optimization and therefore tends to generate smoother class label maps. Generally speaking, iterations of optimization can improve the CA. However, more rounds of iteration will consume more

TABLE IV
CA VALUES (%) OF SAN FRANCISCO AREA DATA WITH DIFFERENT METHODS

Method/Class	Water	Vegetation	High-density urban	Low-density urban	Slanted urban	Overall CA
SVM [5]	86.74	58.67	25.61	60.44	13.67	56.08
Wishart [1]	94.94	89.22	46.28	40.11	60.22	67.43
CNN [11]	100.00	87.67	87.22	84.44	87.78	90.56
MLR _{sub} MLL [58]	100.00	98.13	96.33	36.89	59.11	75.33
KWGraphCNN	100.00	100.00	96.83	100.00	95.89	98.96
SPGraphCNN	100.00	92.31	96.69	99.22	97.67	97.87

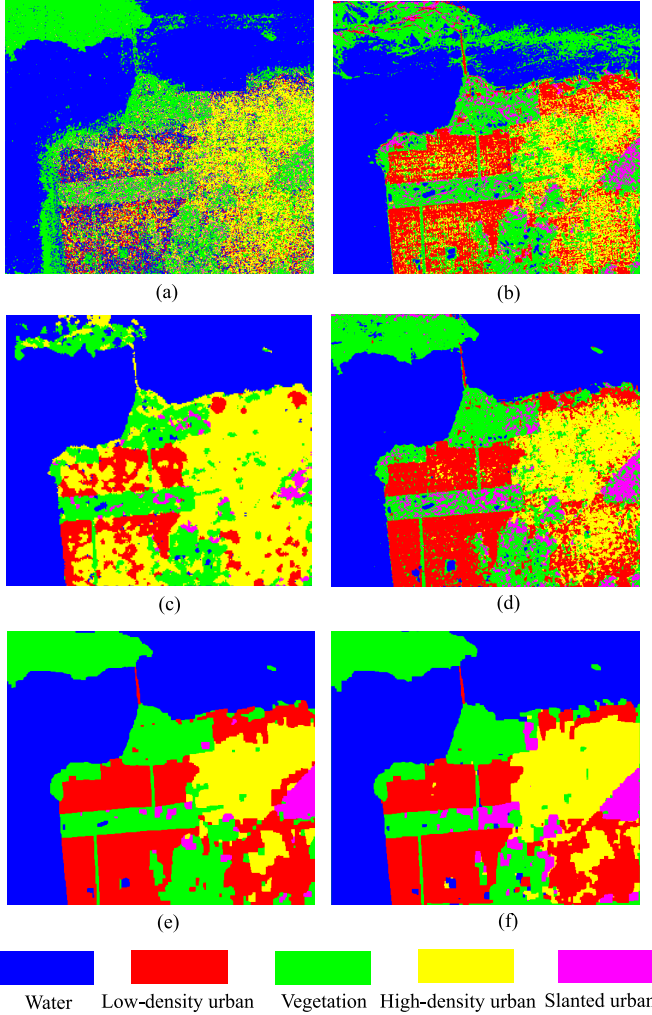


Fig. 6. Classification results of San Francisco area data with different methods. (a) SVM classifier [5]. (b) Wishart classifier [1]. (c) MLR_{sub}MLL classifier [58]. (d) CNN classifier [11]. (e) KWGraphCNN classifier. (f) SPGraphCNN classifier.

computation time and resource. Furthermore, if α_p takes a larger value, too many rounds of optimization will lead to loss of details. Therefore, we next comprehensively analyze the effects of these two parameters to reach a compromise between the CA, computation consumption, and detail preservation.

Fig. 7 shows the classification results taking five typical values of α_p (0.1, 1, 3, 5, 10) after performing for 1–5 iterations of optimization. The *Overall CA* values of iteration 0 correspond to the *K*-Wishart initialization classification result.

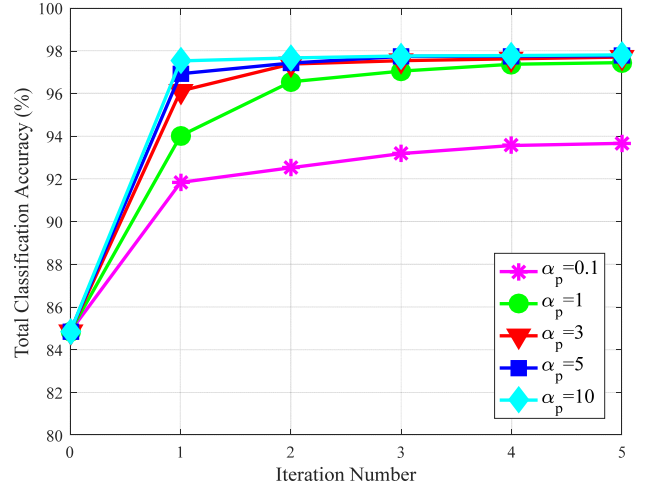


Fig. 7. Influence of pairwise smoothness parameter and iteration number on the *Overall CA* value of the Flevoland area data.

It can be observed from Fig. 7 that the *Overall CA* value taking smaller values of α_p (e.g., 0.1) improves much more slightly than larger α_p and converges at a poorer *Overall CA* value. This is because small α_p value implies weak pairwise smoothness effect, which makes it unable to rectify the wrong classified pixels caused by speckle noise. When α_p gets larger (e.g., ≥ 3), the *Overall CA* improves significantly in the first iteration due to the strong depression of speckle noise and converges at a favorable *Overall CA* after rounds of iteration. For these α_p values, we find that the alternating optimization algorithm roughly converges after three iterations, and more iterations have negligible improvements. Taking both *Overall CA* value and computation consumption into consideration, we take α_p as large values (≥ 3), and the third iteration is preferred as the termination criterion.

To further determine the parameter α_p for the following experiments, Fig. 8 shows the classification results of the Flevoland area data under different α_p values after three rounds of iteration. Fig. 8(a) shows the classification result with $\alpha_p = 3$. Although the whole class label map is much smoother than the original class label map, some specklelike and isolated pixels can still be observed. Fig. 8(b) exhibits the classification result with $\alpha_p = 5$. We can discover from Fig. 8(b) that some wrong classified pixels in Fig. 8(a) have been rectified in Fig. 8(b), such as the island marked by red oval in the top-right part of the image. Moreover, the speckle noise is depressed more than the result shown in Fig. 8(a). Fig. 8(c) displays the classification result with $\alpha_p = 10$.

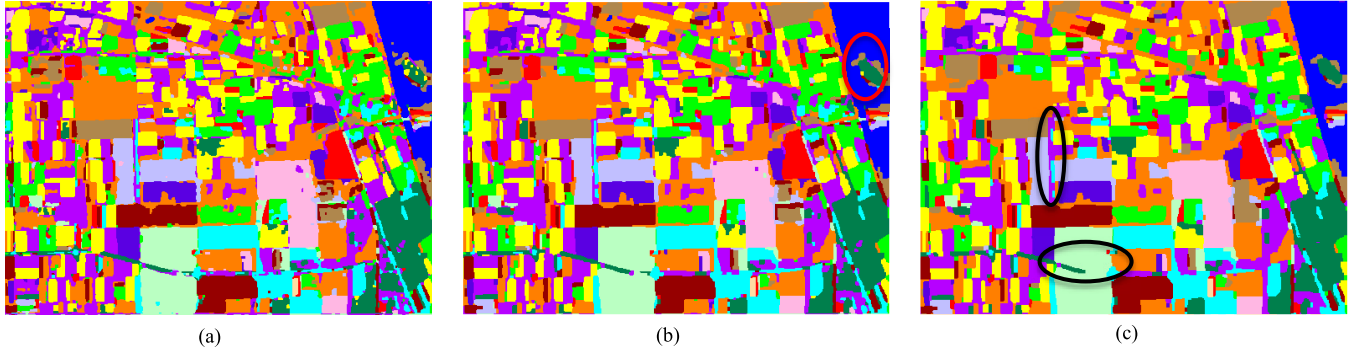


Fig. 8. Classification results under different α_p values after three iterations of optimization for the Flevoland area data. (a) $\alpha_p = 3$. (b) $\alpha_p = 5$. (c) $\alpha_p = 10$.

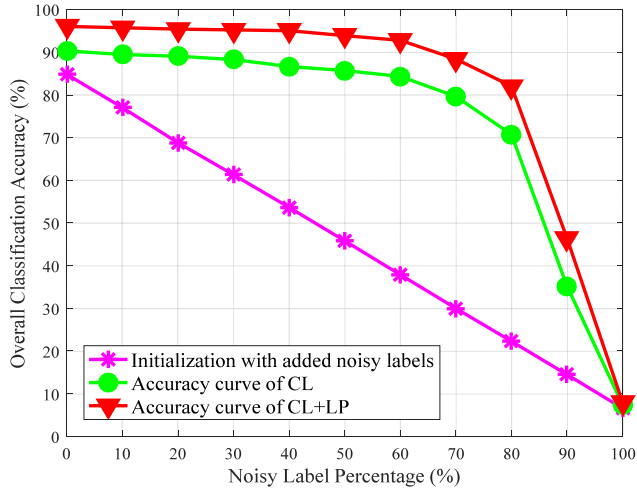


Fig. 9. Impact of noisy labels on *Overall CA* values. These curves represent the classification accuracies with respect to different percentages of noisy labels in the initialized labels.

After three rounds of strong smoothing optimization, the class label map gets smoother than the first two images. However, many tiny details fade out or even disappear in this image. For example, the roads and land boundaries marked in black ovals become shorter or disrupted in Fig. 8(c). To make a balance between the spatial connectivity and detail preservation, $\alpha_p = 5$ is chosen in the following experiments.

However, in applications, α_p can be set according to classification requirement. If results with stronger smoothness are desired, α_p can be set to larger values. If finer details are required, α_p can take smaller values.

2) *Impact of Noisy Labels*: To evaluate the robustness of our proposed method to incorrect input labels, we conduct experiments by adding different percentages of noisy labels on K -Wishart initialized class label map. Given a noise level $s\%$, s percentages of pixels are randomly assigned with labels uniformly sampled from the label set. Fig. 9 displays the *Overall CA* value as a function of the noisy label percentage. The magenta curve represents the *Overall CA* value of initialization with respect to different percentages of noisy labels. The green and red curves indicate the *Overall CA* values of the results after performing CL and CL + LP, respectively.

TABLE V

Overall CA VALUES (%) WITH DIFFERENT BATCH SIZES

Batch Size	10	50	100	200	1000
<i>Overall CA</i>	94.32	97.35	97.75	97.05	95.87

The vertical gap between the accuracy curves of CL and initialization with added noisy labels illustrates the improvement brought by CNN learning, and the gap between the accuracy curves of CL and CL + LP indicates the benefit brought by label propagation. We can observe from Fig. 9 that when the noisy label percentage is lower than 30%, the *Overall CA* value after performing CL keeps roughly consistent with the noise-free case, which demonstrates that CNN learning can effectively correct misclassified pixels under low-noise circumstance. With the increase of noisy label percentage ($\leq 70\%$), the *Overall CA* value marginally decreases. However, the gap between the accuracy curves of CL and initialization with added noisy labels increases dramatically. It reaches the maximum value of 49.74% when the noisy label percentage takes the value of 70%, which illustrates the strong robustness of CNN to noisy labels. When the noisy label percentage increases to a high value ($\geq 80\%$), the *Overall CA* gap gradually decreases. However, even in the highly noisy environments, the *Overall CA* values are still improved considerably, which are 48.55% and 20.78% with noisy label percentage taking values of 80% and 90%, respectively. The accuracy curves of CL and CL + LP keep approximately parallel under different noisy label percentages, which justifies that our graph-based label propagation can stably promote the *Overall CA* values.

3) *Batch Size*: Batch size is an important parameter for CNN optimization. It affects the robustness of gradient computation of the energy function. In general, larger batch size in a reasonable scope increases the memory utilization, reduces the iteration number, and hence speeds up the optimization. Moreover, larger batch size makes the descent direction more accurate and training vibration smaller. However, too large batch size will pose a heavy burden on memory, and the parameter correction speed will get slower. To test the impact of batch size on the proposed method, we conduct experiments with five different batch sizes: 10, 50, 100, 200, and 1000. The *Overall CA* values are shown in Table V.

TABLE VI
Overall CA VALUES (%) WITH DIFFERENT NETWORK DEPTHS

Network Depth	5	6	7	8
Overall CA	97.54	97.75	96.48	95.71

TABLE VII
Overall CA VALUES (%) WITH DIFFERENT KERNEL SIZES

Kernel Size	1	3	5	7
Patch Size	6	8	10	12
Overall CA	97.23	97.75	97.12	96.19

It can be observed from Table V that the *Overall CA* value using a small batch size (e.g., 10) is much lower than a larger batch size (e.g., around 100). This is because the small batch size will cause vibration of the energy function, making the network converge slowly, especially for the network with a large number of classes. When the batch size is set around 100, the *Overall CA* values are very close to each other. However, when the batch size is increased to 1000, the *Overall CA* value is worse compared with the classification results taking 100 as the batch size. We can conclude from Table VI that the best performance is achieved when the batch size is set as 100, which we use as default setting for our experiments.

4) *Network Depth*: To study the effect of different network depths on the classification performance, we conduct experiments on the CNN by reducing or adding fully connected layers. We train and test on four networks with depths 5, 6, 7, and 8, respectively. The experimental results are summarized in Table VI. From Table VI, we can find that increasing the network depth does not always generate better results as a result of the gradient vanishing problem encountered by deeper neural networks. The best performance is achieved by setting the network depth to 6, which we use as default setting.

5) *Kernel Size*: This section investigates the impact of kernel size on the classification performance. We fix the kernel size of the second layer as 2 and evaluate the performance by varying the kernel size of the first convolutional layer. It is worth noting that, with the increase of the kernel size, the input patch size is enlarged correspondingly. Table VII reports the *Overall CA* values with different kernel sizes. It can be observed from Table VII that, when the kernel size increases from 1 to 3, the *Overall CA* value is increased marginally, because more spatial information is incorporated during the network training. However, when the kernel size grows to 5 or 7, the CA degrades. This is because too big kernel size will make the class labels excessively smooth. Moreover, the training time spent on the network training with kernel size 7 is 2.76 times as that of kernel size 3. Comprehensively concluded from the comparison results, kernel size 3 is used as default setting.

6) *Superpixel Size*: For the SPGraphCNN method, superpixel size employed in the initialization process influences the pixel number and the quality of the class label map and further affects the final classification result. To study the impact of superpixel size, we conduct experiments by segmenting the PauliRGB image into different amounts of superpixels. Table VIII presents the *Overall CA* values with respect to

TABLE VIII
Overall CA VALUES (%) WITH DIFFERENT SUPERPIXEL SIZES

Superpixel number	917	5250	21427	47559	100059
Average superpixel size	768	154	38	19	8
Overall CA	93.32	98.18	98.82	98.76	96.92

different superpixel numbers and sizes. The superpixel number indicates the number of superpixels generated using the SLIC algorithm. The average superpixel size is produced by dividing the total pixel number of the image by the superpixel number. We can observe from Table VIII that, when the superpixel size is either too large or too small, the *Overall CA* value of SPGraphCNN is unsatisfactory. This is because if the superpixel size takes a large value (e.g., 768), many pixels are initialized with incorrect labels due to the large-grained superpixel segmentation and hence bring negative impact on the CA. However, if the superpixel size is too small (e.g., 8), only a few pixels are included in each superpixel, making it less contributive to the augmentation of the input label set. We can conclude from Table VIII that the better performance is achieved when the superpixel size takes the value around 20–40.

V. CONCLUSION AND FUTURE WORK

In this paper, we presented a graph-based semisupervised deep learning model for PolSAR image classification. It incorporated feature learning by deep neural network and graph-based class label propagation in a principled framework. Its advantages lie in three points: 1) the proposed semisupervised model utilizes both the labeled and unlabeled pixels in the network training, which achieves a high CA using a very limited number of labeled pixels; 2) the employed CNN automatically extracts discriminative polarimetric features, and the MRF enforces class label smoothness and the alignment of class label boundaries with the image edges; and 3) the proposed iterative and alternating optimization algorithm achieves the state-of-the-art classification results.

In our future work, we plan to investigate the following ideas: 1) instead of utilizing pixelwise patch as input, we will design a network acceleration algorithm with whole image as network input for feature sharing among overlapping patches; 2) we try to devise the MRF to a learnable deep neural network; and 3) instead of relying on the extracted raw features, we are also interested in learning features directly from the input PolSAR image.

REFERENCES

- [1] J.-S. Lee, M. R. Grunes, T. L. Ainsworth, L.-J. Du, D. L. Schuler, and S. R. Cloude, "Unsupervised classification using polarimetric decomposition and the complex Wishart classifier," *IEEE Trans. Geosci. Remote Sens.*, vol. 37, no. 5, pp. 2249–2258, Sep. 1999.
- [2] J. A. Kong, A. A. Swartz, H. A. Yueh, L. M. Novak, and R. T. Shin, "Identification of terrain cover using the optimum polarimetric classifier," *J. Electromagn. Waves Appl.*, vol. 2, no. 2, pp. 171–194, 1988.
- [3] E. Pottier and J. Saillard, "On radar polarization target decomposition theorems with application to target classification, by using neural network method," in *Proc. ICAP*, vol. 1, Apr. 1991, pp. 265–268.
- [4] O. Antropov, R. Rauste, H. Astola, J. Praks, T. Häme, and M. T. Hallikainen, "Land cover and soil type mapping from spaceborne PolSAR data at L-band with probabilistic neural network," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 9, pp. 5256–5270, Sep. 1998.

- [5] C. Lardeux *et al.*, "Support vector machine for multifrequency SAR polarimetric data classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 12, pp. 4143–4152, Dec. 2009.
- [6] S. Fukuda and H. Hirose, "Support vector machine classification of land cover: Application to polarimetric SAR data," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, vol. 1, Jul. 2001, pp. 187–189.
- [7] C. He, S. Li, Z. Liao, and M. Liao, "Texture classification of PolSAR data based on sparse coding of wavelet polarization textures," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 8, pp. 4576–4590, Aug. 2013.
- [8] M. Tao, F. Zhou, Y. Liu, and Z. Zhang, "Tensorial independent component analysis-based feature extraction for polarimetric SAR data classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 5, pp. 2481–2495, May 2015.
- [9] F. Liu, L. Jiao, B. Hou, and S. Yang, "POL-SAR image classification based on Wishart DBN and local spatial information," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 6, pp. 3292–3308, Jun. 2016.
- [10] L. Jiao and F. Liu, "Wishart deep stacking network for fast POLSAR image classification," *IEEE Trans. Image Process.*, vol. 25, no. 7, pp. 3273–3286, Jul. 2016.
- [11] Y. Zhou, H. Wang, F. Xu, and Y.-Q. Jin, "Polarimetric SAR image classification using deep convolutional neural networks," *IEEE Geosci. Remote Sens. Lett.*, vol. 13, no. 12, pp. 1935–1939, Dec. 2016.
- [12] S. R. Cloude and E. Pottier, "A review of target decomposition theorems in radar polarimetry," *IEEE Trans. Geosci. Remote Sens.*, vol. 34, no. 2, pp. 498–518, Mar. 1996.
- [13] A. Freeman and S. L. Durden, "A three-component scattering model for polarimetric SAR data," *IEEE Trans. Geosci. Remote Sens.*, vol. 36, no. 3, pp. 963–973, May 1998.
- [14] Y. Yamaguchi, T. Moriyama, M. Ishido, and H. Yamada, "Four-component scattering model for polarimetric SAR image decomposition," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 8, pp. 1699–1706, Aug. 2005.
- [15] J. S. Lee, D. L. Schuler, R. H. Langz, and K. J. Ransod, "K-distribution for multi-look processed polarimetric SAR imagery," in *Proc. IEEE IGARSS*, vol. 4, Aug. 1994, pp. 2179–2181.
- [16] A. P. Doulgeris, S. N. Anfinsen, and T. Eltoft, "Classification with a non-Gaussian model for PolSAR data," *IEEE Trans. Geosci. Remote Sens.*, vol. 46, no. 10, pp. 2999–3009, Oct. 2008.
- [17] A. P. Doulgeris, "An automatic U-distribution and Markov random field segmentation algorithm for PolSAR images," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 4, pp. 1819–1827, Apr. 2015.
- [18] F. Xu and Y.-Q. Jin, "Deorientation theory of polarimetric scattering targets and application to terrain surface classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 10, pp. 2351–2364, Oct. 2005.
- [19] Y. Wu, K. Ji, W. Yu, and Y. Su, "Region-based classification of polarimetric SAR images using Wishart MRF," *IEEE Geosci. Remote Sens. Lett.*, vol. 5, no. 4, pp. 668–672, Oct. 2008.
- [20] B. Chen, S. Wang, L. Jiao, R. Stolkin, and H. Liu, "A three-component Fisher-based feature weighting method for supervised PolSAR image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 12, no. 4, pp. 731–735, Apr. 2015.
- [21] K. Nigam, A. K. McCallum, S. Thrun, and T. Mitchell, "Text classification from labeled and unlabeled documents using EM," *Mach. Learn.*, vol. 39, nos. 2–3, pp. 103–134, 2000.
- [22] D. Yarowsky, "Unsupervised word sense disambiguation rivaling supervised methods," in *Proc. ACL*, 1995, pp. 189–196.
- [23] A. Blum and T. Mitchell, "Combining labeled and unlabeled data with co-training," in *Proc. COLT*, 1998, pp. 92–100.
- [24] A. Blum and S. Chawla, "Learning from labeled and unlabeled data using graph mincuts," in *Proc. ICML*, 2001, pp. 19–26.
- [25] M. Culp and G. Michailidis, "Graph-based semisupervised learning," *IEEE Trans. Pattern Mach. Intell.*, vol. 30, no. 1, pp. 174–179, Jan. 2008.
- [26] B. Wei, J. Yu, C. Wang, J. Li, and H. Wu, "PolSAR image classification using a semi-supervised classifier based on hypergraph learning," *Remote Sens. Lett.*, vol. 5, no. 4, pp. 386–395, 2014.
- [27] H. Liu, Y. Wang, S. Yang, S. Wang, J. Feng, and L. Jiao, "Large polarimetric SAR data semi-supervised classification with spatial-anchor graph," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 9, no. 4, pp. 1439–1458, Apr. 2016.
- [28] H. Liu, S. Yang, S. Gou, P. Chen, Y. Wang, and L. Jiao, "Fast classification for large polarimetric SAR data based on refined spatial-anchor graph," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 9, pp. 1589–1593, Sep. 2017.
- [29] H. Liu *et al.*, "Semisupervised feature extraction with neighborhood constraints for polarimetric SAR classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 9, no. 7, pp. 3001–3015, Jul. 2016.
- [30] R. Hänsch and O. Hellwich, "Semi-supervised learning for classification of polarimetric SAR-data," in *Proc. IEEE IGARSS*, Jul. 2009, pp. 987–990.
- [31] S. Uhlmann, S. Kiranyaz, and M. Gabbouj, "Semi-supervised learning for ill-posed polarimetric SAR classification," *Remote Sens.*, vol. 6, no. 6, pp. 4801–4830, 2014.
- [32] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [33] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. CVPR*, Jun. 2014, pp. 580–587.
- [34] J. Masci, J. Angulo, and J. Schmidhuber, "A learning framework for morphological operators using counter-harmonic mean," in *Proc. ISMM*, May 2013, pp. 329–340.
- [35] J. Masci, A. Giusti, D. Cireşan, G. Fricout, and J. Schmidhuber, "A fast learning algorithm for image segmentation with max-pooling convolutional networks," in *Proc. ICIP*, Sep. 2013, pp. 2713–2717.
- [36] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, Jul. 2006.
- [37] Y. Bengio, P. Lamblin, D. Popovici, and H. Larochelle, "Greedy layer-wise training of deep networks," in *Proc. NIPS*, vol. 19, 2007, pp. 153–160.
- [38] X. X. Zhu *et al.*, "Deep learning in remote sensing: A comprehensive review and list of resources," *IEEE Geosci. Remote Sens. Mag.*, vol. 5, no. 4, pp. 8–36, Dec. 2017.
- [39] L. Zhang, L. Zhang, and B. Du, "Deep learning for remote sensing data: A technical tutorial on the state of the art," *IEEE Geosci. Remote Sens. Mag.*, vol. 4, no. 2, pp. 22–40, Jun. 2016.
- [40] Y. Wei, Q. Yuan, H. Shen, and L. Zhang, "Boosting the accuracy of multispectral image pansharpening by learning a deep residual network," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 10, pp. 1795–1799, Oct. 2017.
- [41] Q. Yuan, Y. Wei, X. Meng, H. Shen, and L. Zhang, "A multiscale and multi-depth convolutional neural network for remote sensing imagery pan-sharpening," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 3, pp. 978–989, Mar. 2018.
- [42] G. D. De Grandi, J.-S. Lee, and D. L. Schuler, "Target detection and texture segmentation in polarimetric SAR images using a wavelet frame: Theoretical aspects," *IEEE Trans. Geosci. Remote Sens.*, vol. 45, no. 11, pp. 3437–3453, Nov. 2007.
- [43] K. S. Chen, W. P. Huang, D. H. Tsay, and F. Amar, "Classification of multifrequency polarimetric SAR imagery using a dynamic learning neural network," *IEEE Trans. Geosci. Remote Sens.*, vol. 34, no. 3, pp. 814–820, Mar. 1996.
- [44] A. Masjedi, M. J. V. Zoj, and Y. Maghsoudi, "Classification of polarimetric SAR images based on modeling contextual information and using texture features," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 2, pp. 932–943, Feb. 2016.
- [45] L. Zhang, W. Ma, and D. Zhang, "Stacked sparse autoencoder in PolSAR data classification using local spatial information," *IEEE Geosci. Remote Sens. Lett.*, vol. 13, no. 9, pp. 1359–1363, Sep. 2016.
- [46] W. Xie, L. Jiao, and J. Zhao, "PolSAR image classification via D-KSVD and NSCT-domain features extraction," *IEEE Geosci. Remote Sens. Lett.*, vol. 13, no. 2, pp. 227–231, Feb. 2016.
- [47] S. T. Tu, J. Y. Chen, W. Yang, and H. Sun, "Laplacian eigenmaps-based polarimetric dimensionality reduction for SAR image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 1, pp. 170–179, Jan. 2012.
- [48] H. Bi, L. Xu, X. Cao, and Z. Xu, "Polar image classification based on three-dimensional wavelet texture features and Markov random field," in *Proc. IEEE IGARSS*, Jul. 2017, pp. 3921–3928.
- [49] S. R. Cloude and E. Pottier, "An entropy based classification scheme for land application of polarimetric SAR," *IEEE Trans. Geosci. Remote Sens.*, vol. 35, no. 1, pp. 68–78, Jan. 1997.
- [50] H. Bi, J. Sun, and Z. Xu, "Unsupervised PolSAR image classification using discriminative clustering," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 6, pp. 3531–3544, Jun. 2017.
- [51] Y. Y. Boykov and M.-P. Jolly, "Interactive graph cuts for optimal boundary & region segmentation of objects in N-D images," in *Proc. ICCV*, Jul. 2001, pp. 105–112.

- [52] D. Lin, J. Dai, J. Jia, K. He, and J. Sun, "ScribbleSup: Scribble-supervised convolutional networks for semantic segmentation," in *Proc. CVPR*, 2016, pp. 3159–3167.
- [53] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning internal representations by error propagation," in *Neurocomputing: Foundations of Research*. Cambridge, MA, USA: MIT Press, 1988, pp. 318–362.
- [54] M. F. Tappen and W. T. Freeman, "Comparison of graph cuts with belief propagation for stereo, using identical MRF parameters," in *Proc. ICCV*, 2003, pp. 900–906.
- [55] C. Zhu, R. H. Byrd, P. Lu, and J. Nocedal, "Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization," *ACM Trans. Math. Softw.*, vol. 23, no. 4, pp. 550–560, Dec. 1997.
- [56] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk, "SLIC superpixels compared to state-of-the-art superpixel methods," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012.
- [57] J.-S. Lee, M. R. Grunes, and G. de Grandi, "Polarimetric SAR speckle filtering and its implication for classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 37, no. 5, pp. 2363–2373, Sep. 1999.
- [58] J. Li, J. M. Bioucas-Dias, and A. Plaza, "Spectral-spatial hyperspectral image segmentation using subspace multinomial logistic regression and Markov random fields," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 3, pp. 809–823, Mar. 2012.
- [59] K. He and J. Sun, "Convolutional neural networks at constrained time cost," in *Proc. CVPR*, 2014, pp. 5353–5360.



Haixia Bi received the B.S. and M.S. degrees from the Ocean University of China, Qingdao, China, in 2003 and 2006, respectively. She is currently pursuing the Ph.D. degree with Xi'an Jiaotong University, Xi'an, China.

Her research interests include remote sensing image processing and machine learning algorithms.



Jian Sun (M'08) received the B.S. degree from the University of Electronic Science and Technology of China, Chengdu, China, in 2003, and the Ph.D. degree in applied mathematics from Xi'an Jiaotong University, Xi'an, China, in 2009.

He was a Visiting Student with Microsoft Research Asia, Beijing, China, from 2005 to 2008, a Post-Doctoral Researcher with the University of Central Florida, Orlando, FL, USA, from 2009 to 2010, and a Post-Doctoral Researcher with the Willow Project Team, École Normale Supérieure de Paris and INRIA, Paris, France, from 2012 to 2014. He is currently a Professor with the School of Mathematics and Statistics, Xi'an Jiaotong University. His research interests include computer vision, medical image analysis, and machine learning.



Zongben Xu received the Ph.D. degree in mathematics from Xi'an Jiaotong University, Xi'an, China, in 1987.

He currently serves as the Chief Scientist of the National Basic Research Program of China (973 Project) and the Director of the Institute for Information and System Sciences, Xi'an Jiaotong University, where he is a member of the Chinese Academy of Sciences, Beijing, China. His research interests include intelligent information processing and applied mathematics.

Dr. Xu was a recipient of the National Natural Science Award of China in 2007 and the CSIAM Su Buchin Applied Mathematics Prize in 2008. He delivered a 45 min-talk on the International Congress of Mathematicians in 2010.